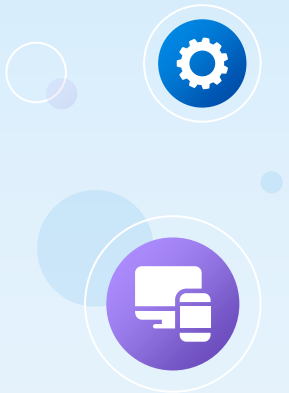
CSE212 Theory of Computation and Compilers

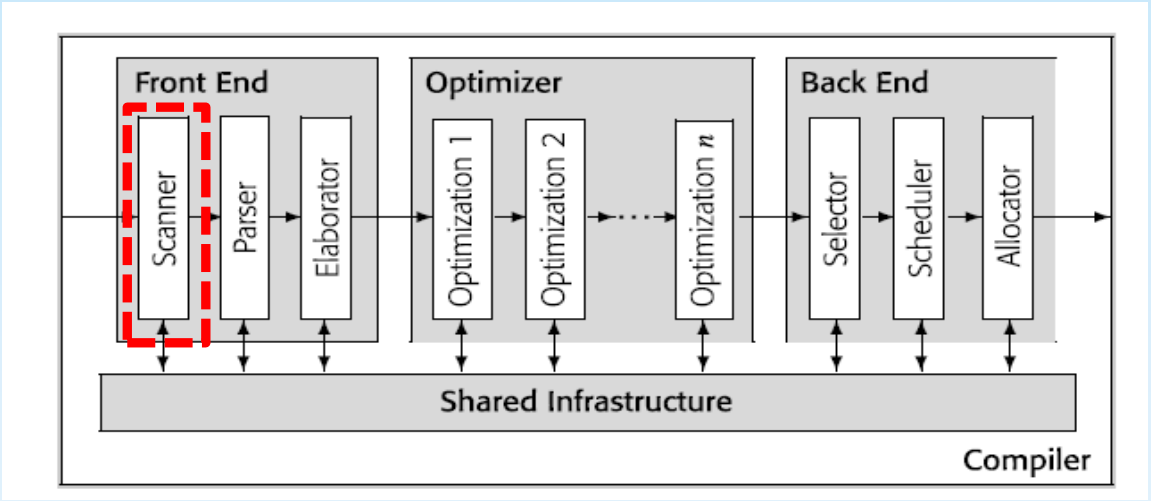
Scanners

Sara El-Metwally,
Associate Professor,
Computer Science Department

Table of contents



Compiler Design



Scanner

- The scanner's task is to transform a stream of characters into a stream of words in the input language.
- Each word must be classified into a syntactic category, or "part of speech."
- The scanner, or lexical analyzer, reads a stream of characters and produces a stream of words.
- The parser, or syntax analyzer, fits that stream of words to a grammatical model of the input language.
- The scanner aggregates characters into words. For each word, it determines if the word is valid in the source language.
- For each valid word, it assigns the word a syntactic category, or part of speech.

Scanner

❑ To translate a program, the compiler must understand both its lexical structure—the spellings of the words in the program—and its syntactic structure—the grammatical way that words fit together to form statements and programs.

❑ Each time it is called, the scanner produces a pair, (lexeme, category), where lexeme is the spelling of the word and category is its syntactic category. This pair is sometimes called a token.

❑ To find and classify words, the scanner applies a set of rules that describe the lexical structure of the input programming language, sometimes called microsyntax.

Scanner

❑ At design time, the compiler writer creates specifications for the microsyntax of the programming language (the spelling of words). The compiler writer chooses an implementation method and writes any code that is needed.

❑ At build time, the compiler writer invokes tools that build the actual executable scanner.

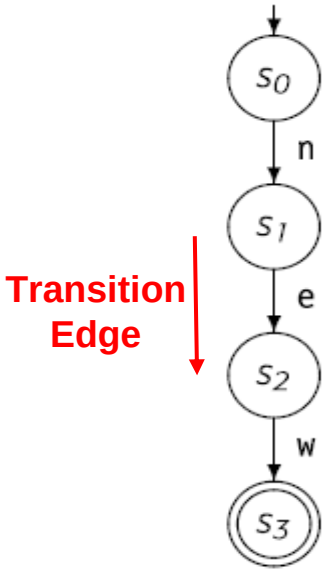
❑ At compile time, the end user invokes the compiler to translate application code into executable code. The compiler includes the scanner; it uses the scanner to convert the application code from a string of characters into a string of words, and to classify each of those words into a syntactic category or part of speech.

Recognizing Words

Consider the problem of recognizing the keyword new

```
c ← NextChar();
if (c = 'n') then
    c ← NextChar();
    if (c = 'e') then
        c ← NextChar();
        if (c = 'w') then
            report success;
        else
            try something else;
    else
        try something else;
else
    try something else;
```

(a) Code



(b) Recognizer

Start State

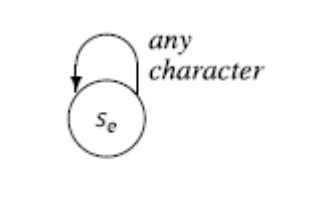
Transition Diagram

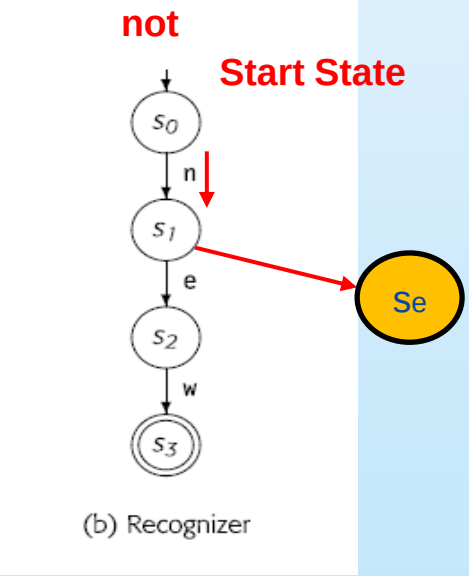
Transition Edge

Accepting State

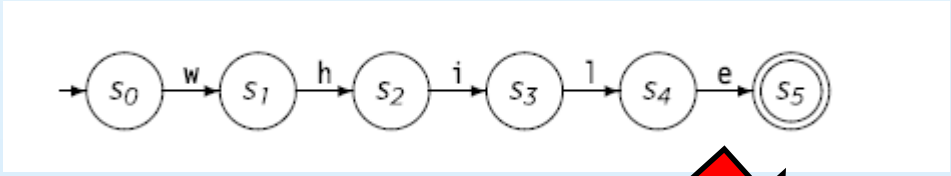
Scanner

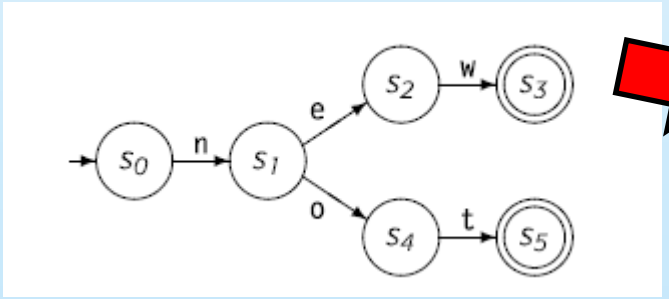
❑ When we draw the transition diagram of a recognizer, we usually omit transitions to the error state. Each state has an implicit transition to the error state on each unspecified input.

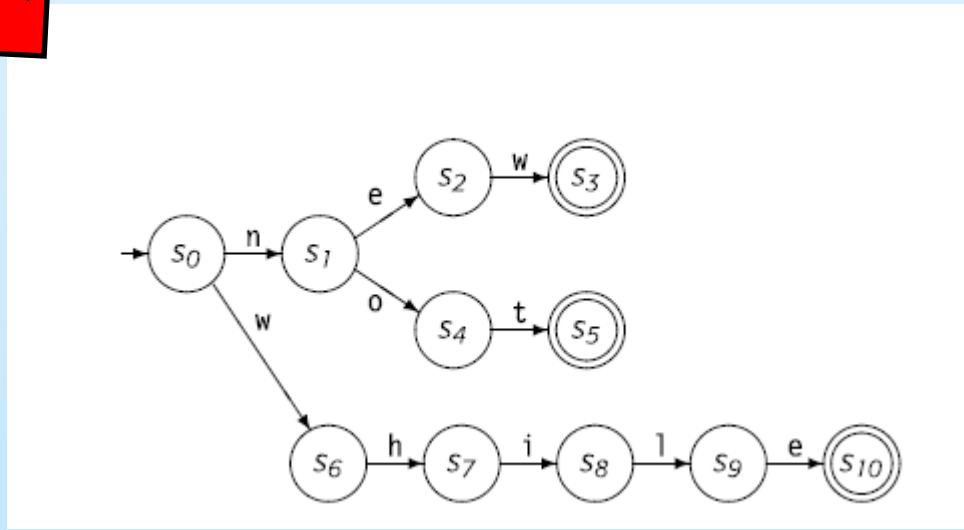




a recognizer for keyword while, new, not





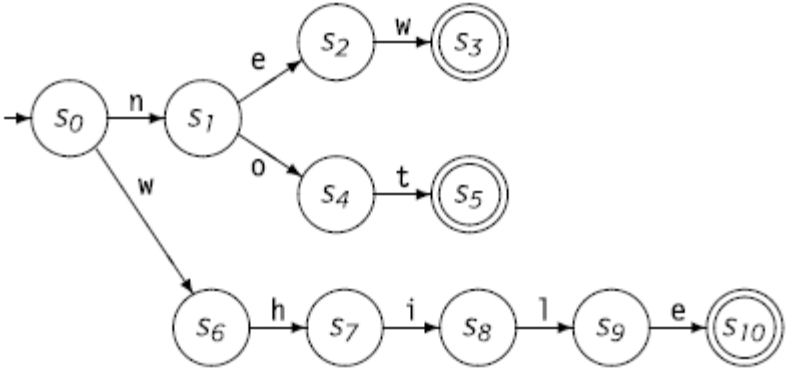


A Formalism for Recognizers

- Transition diagrams can be viewed as formal mathematical objects called finite automata, that specify recognizers.
- Formally, a finite automaton (FA) is a five-tuple $(S, \Sigma, \delta, s_0, S_A)$.

- S is the finite set of states in the recognizer, including s_e.
- Σ is the finite alphabet used by the recognizer. Typically, Σ is the union of the edge labels in the transition diagram.
- $\delta(s, c)$ is the recognizer's transition function. It maps each state $s \in S$ and each character $c \in \Sigma$ into some next state. In state s_i with input character c, the FA takes the transition $s_i \xrightarrow{c} \delta(s_i, c)$.
- $s_0 \in S$ is the designated start state.
- S_A is the set of accepting states, $S_A \subseteq S$. Each state in S_A appears as a double circle in the transition diagram.

Formalize the following FA



$$S = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_e\}$$

$$\Sigma = \{\mathtt{e}, \mathtt{h}, \mathtt{i}, \mathtt{l}, \mathtt{n}, \mathtt{o}, \mathtt{t}, \mathtt{w}\}$$

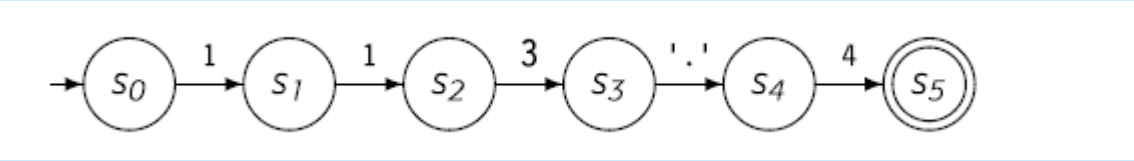
$$\delta = \left\{ \begin{array}{ccccc} s_0 \xrightarrow{\mathtt{n}} s_1, & s_0 \xrightarrow{\mathtt{w}} s_6, & s_1 \xrightarrow{\mathtt{e}} s_2, & s_1 \xrightarrow{\mathtt{o}} s_4, & s_2 \xrightarrow{\mathtt{w}} s_3, \\ s_4 \xrightarrow{\mathtt{t}} s_5, & s_6 \xrightarrow{\mathtt{h}} s_7, & s_7 \xrightarrow{\mathtt{i}} s_8, & s_8 \xrightarrow{\mathtt{l}} s_9, & s_9 \xrightarrow{\mathtt{e}} s_{10} \end{array} \right\}$$

$$s_0 = s_0$$

$$S_A = \{s_3, s_5, s_{10}\}$$

FA

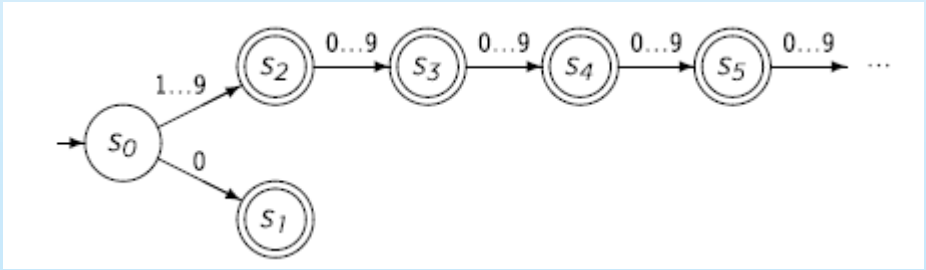
❑ FA accepts a string x if and only if, starting in s0, the sequence of characters in x takes the FA through a series of transitions that leaves it in an accepting state when the entire string has been consumed. This corresponds to our intuition for the transition diagram.

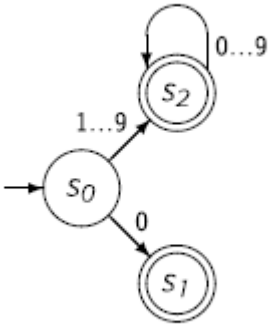


Which ---- is recognized by the above FA?

FA

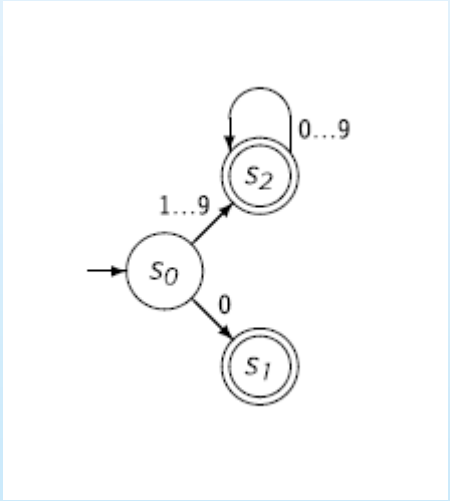
❑ An integer is either zero, or it is a series of one or more digits where the first digit is from one to nine, and the subsequent digits are from zero to nine. (This definition rules out leading zeros.) How would we draw a transition diagram for this definition?





The cyclic transition diagram

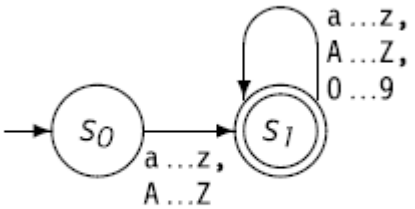
Transition Table



δ	0	1	2	3	4	5	6	7	8	9	Other
s_0	s_1	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_e
s_1	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e
s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_2	s_e
s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e	s_e

FA for identifier

❑ an identifier consists of an alphabetic character followed by zero or more alphanumeric characters.



FA for Identifiers

Regular Expressions

❑ The set of words accepted by a finite automaton, F, forms a language, denoted L(F).

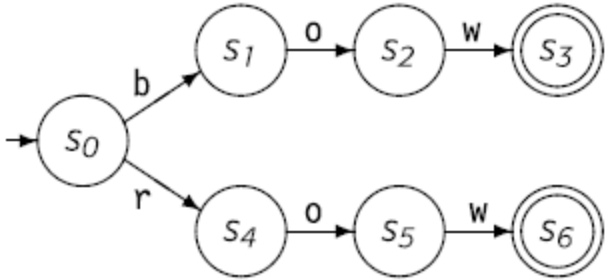
❑ The transition diagram of F specifies, in precise detail, how to spell every word in that language.

❑Transition diagrams can be complex and non-intuitive.Thus, most systems use a notation called a regular expression (RE) to describe spelling. Any language described by an RE is considered a regular language.

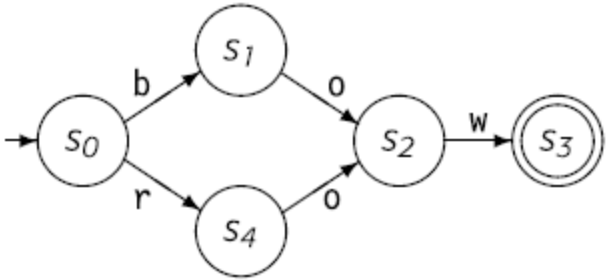
❑ The language consisting of the single word bow can be described by an RE written as bow.

❑ bow | row and (b | r) ow both specify the same language in RE.

Regular Expressions



(a) FA suggested by *bow* | *row*



(b) FA suggested by (*b* | *r*) *ow*

Regular Expressions

❑ For the RE x, we write this as x*, with the meaning “zero or more occurrences of x.” We call the * operator Kleene closure, or closure for short.

❑ an unsigned integer is either a zero, or a nonzero digit followed by zero or more digits.

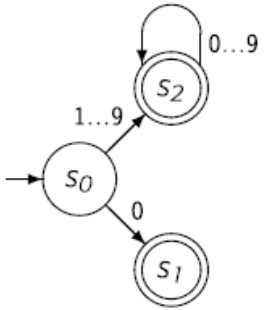
❑ 0 | (1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9) (0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9)*.

❑ 0 | ([1. . . 9]) ([0. . . 9])*.

Complement

The RE $^{\wedge}c$ specifies the set $\{\Sigma - c\}$, the complement of c with respect to Σ.

Complement has higher precedence than *, |, or $^{+}$.



FA for Unsigned Integers

Regular Expressions

2.3.1 Formalizing the Notation

To work with REs in a rigorous way, we need a formal definition. Assume that we have an alphabet, Σ. An RE describes a set of strings over the characters in Σ, plus an additional character ϵ that represents the empty string. The set of strings defined by an RE is called a *language*. We denote the language described by some RE r as $L(r)$.

An RE is built up from three basic operations:

1. *Alternation* The alternation, or union, of two sets of REs, r and s, denoted $r \mid s$, is $\{x \mid x \in L(r) \text{ or } x \in L(s)\}$.
2. *Concatenation* The concatenation of two REs r and s, denoted rs, contains all strings formed by prepending a string from $L(r)$ onto one from $L(s)$, or $\{xy \mid x \in L(r) \text{ and } y \in L(s)\}$.
3. *Closure* The Kleene closure of r, denoted r^*, is $\bigcup_{i=0}^{\infty} r^i$. $L(r^*)$ contains all strings that consist of zero or more words from $L(r)$.

Regular Expressions

For convenience, we sometimes use a *finite closure*. The notation r^i, $i \geq 0$, denotes from one to i occurrences of r. A finite closure can always be replaced with an enumeration of the possibilities; for example, r^3 is just $(\epsilon \mid r \mid rr \mid rrr)$ The *positive closure*, denoted r^+, is just rr^* and consists of one or more occurrences of r. Since the finite and positive closures can be rewritten in terms of alternation, concatenation, and Kleene closure

Regular Expressions

Language Rule	Regular Expression
an alphabetic character followed by zero or more alphanumeric characters.	$([A\ldots Z] \mid [a\ldots z])\ ([A\ldots Z] \mid [a\ldots z] \mid [0\ldots 9])^*$
Most languages also allow a few special characters, such as _, %, $, or & in identifiers.	?
If the language limits the length of an identifier, we can use a finite closure	$([A\ldots Z] \mid [a\ldots z])\ ([A\ldots Z] \mid [a\ldots z] \mid [0\ldots 9])^5$
real numbers	$(0 \mid [1\ldots 9]\ [0\ldots 9]^*)\ (\epsilon \mid .\ [0\ldots 9]^*)$
More on Page 39-40 Book	

Regular Expressions

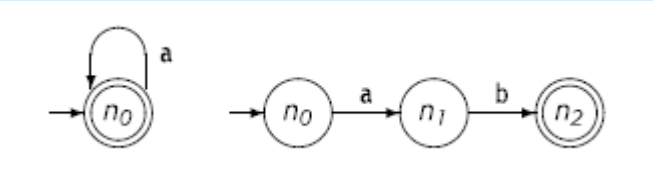
```
[_a-zA-Z][_a-zA-Z0-9]{0,30}
```

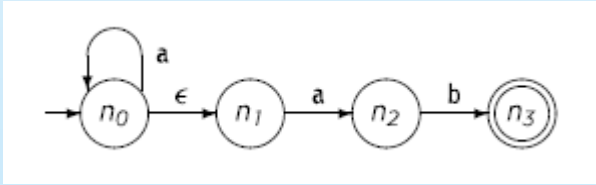
an alphabetic character or underscore followed by X alphanumeric characters or underscore where 0<=X<=30 .

NFA vs. DFA

❑ Nondeterministic FA (**NFA**): an FA where the transition function can be multivalued and can include ϵ-transitions.

❑ Deterministic FA (**DFA**): an FA where the transition function is single-valued and does not include ϵ-transitions.





Input : a, ab, aab

Transitions:

$n_0 \xrightarrow{a} n_0$

$n_0 \xrightarrow{\epsilon} n_1, n_1 \xrightarrow{a} n_2$

NFA vs. DFA

Deterministic Finite Automata

↓

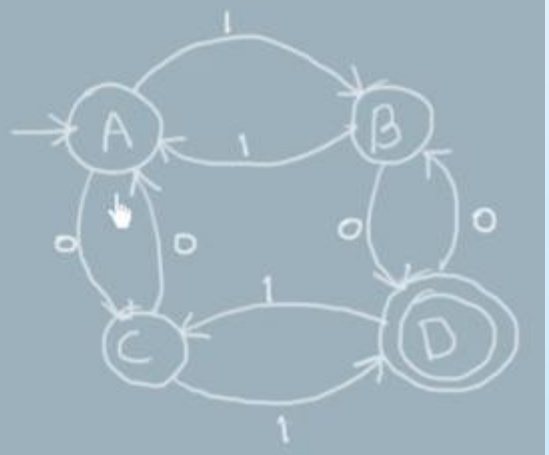
DETERMINISM

>> In DFA, given the current state
we know what the next state will be

>> It has only one unique next state

>> It has no choices or randomness

>> It is simple and easy to design



NFA vs. DFA

Non-deterministic Finite Automata

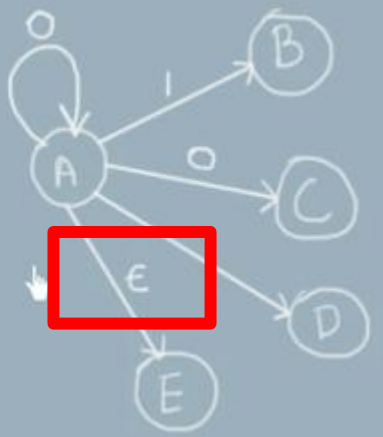
↓

NON-DETERMINISM

>> In NFA, given the current state there could be multiple next states

>> The next state may be chosen at random

>> All the next states may be chosen in parallel



Thanks!